Source code

Import Required Libraries

```
numpy

pandas

matplotlib

scikit-learn

joblib

tensorflow>=2.0  # optional; required only if you want LSTM

shap              # optional; only if you want SHAP explainability
```

Description of Code

```
# Predictive Modeling System for Efficient Power Management

This repository contains a single-file implementation
energy_forecasting_pipeline.py which:

- Loads or generates a synthetic dataset of store-wise energy consumption.

- Preprocesses and engineers features (lags, rolling mean, time features).

- Trains a Random Forest model (fast baseline) and optionally an LSTM model
(requires TensorFlow).

- Evaluates models (MAE, RMSE, R2, MAPE), saves models and scaler.

- Produces plots and simple optimization recommendations.

- Optional SHAP explainability for Random Forest (if shap installed).


## Quick start

1. Install dependencies: pip install -r requirements.txt

2. (Optional) Put power_data.csv with columns: timestamp, store_id, energy_kwh,
temperature, humidity, footfall, appliance_load, dayofweek

3. Run: python energy_forecasting_pipeline.py

4. Outputs in ./outputs/ (models, plots, recommendations)


## Notes

- For production: improve per-store LSTM sequence handling, perform
hyperparameter tuning, add retraining pipeline, secure model deployment.
```

Main Source Code

```
"""
energy_forecasting_pipeline.py
Predictive Modeling System for Efficient Power Management
Single-file collective source code: data generation/loading, preprocessing, models (RF + optional LSTM),
evaluation, SHAP explainability (optional), optimization suggestions, plotting, saving, and inference helpers.

Usage:
    - Put your CSV named "power_data.csv" in same folder OR let the script generate synthetic data.
    - Run: python energy_forecasting_pipeline.py
    - Outputs: saved models (rf_energy_model.joblib, scaler.joblib), optional LSTM, plots in ./outputs/

Author: Keshav Kumar
Date: 2025-XX-XX
"""

import os
import math
import argparse
from datetime import datetime, timedelta
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# sklearn imports
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
import joblib

# Optional imports for LSTM and SHAP
try:
    import tensorflow as tf
    from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import LSTM, Dense
    from tensorflow.keras.callbacks import EarlyStopping
    TF_AVAILABLE = True
except Exception:
    TF_AVAILABLE = False

try:
    import shap
    SHAP_AVAILABLE = True
except Exception:
    SHAP_AVAILABLE = False

# ----------------------------
# 1) Data generation / loading
# ----------------------------
def generate_synthetic_data(n_stores=3, days=60, seed=42):
    """
    Generates hourly synthetic multistore dataset for 'days' days.
    Returns pd.DataFrame with columns:
      timestamp, store_id, energy_kwh, temperature, humidity, footfall,
appliance_load, dayofweek
    """
    np.random.seed(seed)
    periods = int(24 * days)
    base_time = datetime(2024, 1, 1)
    rows = []
    for store in range(1, n_stores + 1):
        store_effect = (store - 1) * 20
        for t in range(periods):
```

```
            ts = base_time + timedelta(hours=t)
            hour = ts.hour
            dow = ts.weekday()
            # daily pattern (simple)
            daily_season = 20 * (math.sin((hour - 12) / 24 * 2 * math.pi) + 1)
            weekly = 10 if dow >= 5 else 0
            temp = 25 + 6 * np.sin((t / (24 * 7)) * 2 * math.pi) +
np.random.normal(0, 1)
            humidity = 50 + 8 * np.cos((t / 24) * 2 * math.pi / 7) +
np.random.normal(0, 2)
            footfall = max(0, int(100 + 40 * math.sin((hour - 13) / 24 * 2 *
math.pi) + np.random.normal(0, 8)))
            appliance_load = 5 + 0.2 * footfall + np.random.normal(0, 2)
            base = 50 + store_effect + daily_season + weekly
            energy = base + 0.35 * footfall + 0.5 * appliance_load + 0.25 *
temp + np.random.normal(0, 4)
            rows.append([ts, f"S{store:02d}", round(energy, 3), round(temp, 2),
round(humidity, 2),
                         int(footfall), round(appliance_load, 2), dow])
    df = pd.DataFrame(rows, columns=["timestamp", "store_id", "energy_kwh",
"temperature",
                                     "humidity", "footfall", "appliance_load",
"dayofweek"])
    return df
def load_or_generate(csv_path="power_data.csv", generate_if_missing=True):
    """
    Tries to load CSV; if not found and generate_if_missing=True, creates
synthetic dataset.
    """
    if os.path.exists(csv_path):
        print(f"[INFO] Loading dataset from {csv_path}")
        df = pd.read_csv(csv_path, parse_dates=["timestamp"])
    else:
        if generate_if_missing:
            print("[INFO] CSV not found. Generating synthetic dataset.")
            df = generate_synthetic_data(n_stores=3, days=60)
        else:
```

```
            raise FileNotFoundError(f"{csv_path} not found.")
    df['timestamp'] = pd.to_datetime(df['timestamp'])
    return df



# ----------------------------
# 2) Preprocessing & features
# ----------------------------
def prepare_features(df, lag_hours=[1, 24, 168], drop_na=True):
    """
    Create time features, lag features, rolling features, and encode store_id.
    Returns DataFrame with features ready for modeling (with timestamp
preserved).
    """
    df = df.sort_values(["store_id", "timestamp"]).reset_index(drop=True)
    # time features
    df['hour'] = df['timestamp'].dt.hour
    df['day'] = df['timestamp'].dt.day
    df['month'] = df['timestamp'].dt.month
    df['is_weekend'] = df['dayofweek'].apply(lambda x: 1 if x >= 5 else 0)

    # Lag features by store
    for lag in lag_hours:
        df[f'lag_{lag}'] = df.groupby('store_id')['energy_kwh'].shift(lag)

    # Rolling mean (24-hour)
    df['rolling_24_mean'] =
df.groupby('store_id')['energy_kwh'].transform(lambda x: x.rolling(window=24,
min_periods=1).mean())

    # Drop start rows with NaN (due to lags)
    if drop_na:
        df = df.dropna().reset_index(drop=True)

    # One-hot encode store_id (drop_first=True to avoid multicollinearity)
```

```
    df = pd.get_dummies(df, columns=['store_id'], drop_first=True)

    return df


# ----------------------------
# 3) Splitting and scaling
# ----------------------------
def time_based_train_test_split(df, test_ratio=0.2):
    """
    Splits chronologically (important for time series).
    """
    df = df.sort_values('timestamp').reset_index(drop=True)
    split_idx = int(len(df) * (1 - test_ratio))
    train = df.iloc[:split_idx].copy()
    test = df.iloc[split_idx:].copy()
    return train, test


def scale_features(train_X, test_X):
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(train_X)
    X_test_scaled = scaler.transform(test_X)
    return X_train_scaled, X_test_scaled, scaler


# ----------------------------
# 4) Metrics
# ----------------------------
def compute_metrics(y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = math.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    with np.errstate(divide='ignore', invalid='ignore'):
```

```
        mape = np.mean(np.abs((y_true - y_pred) / (y_true + 1e-6))) * 100
    return {'MAE': float(mae), 'RMSE': float(rmse), 'R2': float(r2), 'MAPE':
float(mape)}



# ------------------------------
# 5) Random Forest model
# ------------------------------
def train_random_forest(X_train, y_train, n_estimators=100, random_state=42):
    model = RandomForestRegressor(n_estimators=n_estimators,
random_state=random_state, n_jobs=-1)
    model.fit(X_train, y_train)
    return model



# ------------------------------
# 6) LSTM model (optional)
# ------------------------------
def create_sequences_for_lstm(df_features, feature_cols,
target_col='energy_kwh', seq_len=24):
    """
    Create sequences for LSTM. df_features must be sorted chronologically.
    Returns X_seq, y_seq numpy arrays.
    """
    arr = df_features[feature_cols + [target_col]].values
    Xs, ys = [], []
    for i in range(seq_len, len(arr)):
        Xs.append(arr[i-seq_len:i, :-1])
        ys.append(arr[i, -1])
    return np.array(Xs), np.array(ys)

def build_lstm_model(input_shape):
    model = Sequential()
    model.add(LSTM(64, input_shape=input_shape, return_sequences=False))
    model.add(Dense(32, activation='relu'))
```

```
    model.add(Dense(1))
    model.compile(optimizer='adam', loss='mse')
    return model
# ----------------------------
# 7) Optimization recommendation
# ----------------------------
def optimization_recommendation(predictions, threshold_factor=1.15):
    """
    Simple rule-based recommendations:
    - If predicted > threshold_factor * mean_pred => 'High demand'
recommendation
    - Else => 'Normal'
    Returns list of dicts
    """
    mean_pred = np.mean(predictions)
    threshold = mean_pred * threshold_factor
    recs = []
    for idx, val in enumerate(predictions):
        if val > threshold:
            recs.append({'index': idx, 'predicted': float(val),
                         'recommendation': 'High demand predicted -> shift non-
critical loads to off-peak; reduce HVAC or dim lighting'})
        else:
            recs.append({'index': idx, 'predicted': float(val),
'recommendation': 'Normal'})
    return recs



# ----------------------------
# 8) Explainability using SHAP (optional)
# ----------------------------
def compute_shap_for_rf(rf_model, X_sample, feature_names, out_dir="outputs"):
    if not SHAP_AVAILABLE:
        print("[WARN] SHAP not available. Install shap to compute
explainability.")
        return None
```

```
    explainer = shap.Explainer(rf_model.predict, X_sample)
    shap_values = explainer(X_sample)
    # Summary plot
    os.makedirs(out_dir, exist_ok=True)
    shap.summary_plot(shap_values, features=X_sample,
feature_names=feature_names, show=False)
    plt.tight_layout()
    plt.savefig(os.path.join(out_dir, "shap_summary_rf.png"))
    plt.close()
    print("[INFO] SHAP summary plot saved:", os.path.join(out_dir,
"shap_summary_rf.png"))
    return shap_values


# ----------------------------
# 9) Plot helpers
# ----------------------------
def plot_actual_vs_pred(y_true, y_pred, title, out_path=None, max_points=500):
    plt.figure(figsize=(10,4))
    plt.plot(y_true[:max_points], label='Actual', marker='o', linewidth=1)
    plt.plot(y_pred[:max_points], label='Predicted', marker='x', linewidth=1)
    plt.title(title)
    plt.xlabel('Sample index')
    plt.ylabel('Energy (kWh)')
    plt.legend()
    plt.grid(True)
    if out_path:
        plt.tight_layout()
        plt.savefig(out_path)
        plt.close()
        print("[INFO] Saved plot:", out_path)
    else:
        plt.show()
```

```
def plot_error_histogram(y_true, y_pred, out_path=None):
    errors = y_true - y_pred
    plt.figure(figsize=(6,4))
    plt.hist(errors, bins=30)
    plt.title('Error Distribution')
    plt.xlabel('Error (Actual - Predicted)')
    plt.ylabel('Frequency')
    plt.grid(True)
    if out_path:
        plt.tight_layout()
        plt.savefig(out_path)
        plt.close()
        print("[INFO] Saved error histogram:", out_path)
    else:
        plt.show()



# ----------------------------
# 10) Main pipeline
# ----------------------------
def run_pipeline(csv_path="power_data.csv", use_lstm=True,
output_dir="outputs"):
    os.makedirs(output_dir, exist_ok=True)

    # Load or generate
    df = load_or_generate(csv_path=csv_path, generate_if_missing=True)
    print("[INFO] Raw data rows:", len(df))

    # Feature preparation
    df_feat = prepare_features(df, lag_hours=[1, 24, 168])
    print("[INFO] Features prepared. Rows after feature creation:",
len(df_feat))

    # Feature list
    exclude = ['timestamp', 'energy_kwh']
```

```
    feature_cols = [c for c in df_feat.columns if c not in exclude]
    target_col = 'energy_kwh'

    # Time-based split
    train_df, test_df = time_based_train_test_split(df_feat, test_ratio=0.2)
    X_train = train_df[feature_cols].values
    y_train = train_df[target_col].values
    X_test = test_df[feature_cols].values
    y_test = test_df[target_col].values

    # Scale features
    X_train_s, X_test_s, scaler = scale_features(X_train, X_test)

    # Train Random Forest
    print("[INFO] Training Random Forest...")
    rf = train_random_forest(X_train_s, y_train, n_estimators=100)
    y_pred_rf = rf.predict(X_test_s)
    rf_metrics = compute_metrics(y_test, y_pred_rf)
    print("[RESULT] Random Forest metrics:", rf_metrics)

    # Save RF and scaler
    joblib.dump(rf, os.path.join(output_dir, "rf_energy_model.joblib"))
    joblib.dump(scaler, os.path.join(output_dir, "scaler.joblib"))
    print("[INFO] Saved RF model and scaler in", output_dir)

    # Plot RF results
    plot_actual_vs_pred(y_test, y_pred_rf, "Actual vs Predicted (Random
Forest)", out_path=os.path.join(output_dir, "rf_actual_vs_pred.png"))
    plot_error_histogram(y_test, y_pred_rf, out_path=os.path.join(output_dir,
"rf_error_hist.png"))

    shap_vals = None
    if SHAP_AVAILABLE:
        # Use a sample of test data for SHAP (small for speed)
        sample_X = X_test_s[:500]
```

```
        compute_shap_for_rf(rf, sample_X, feature_names=feature_cols,
out_dir=output_dir)


    lstm_metrics = None

    if use_lstm and TF_AVAILABLE:

        print("[INFO] Preparing sequences for LSTM...")

        seq_len = 24

        # For LSTM we create sequences from training and testing concatenated
per chronological order.

        # WARNING: This simple approach concatenates multiple stores — for
production you may build per-store sequences.

        train_arr = train_df[feature_cols + [target_col]].values

        test_arr = test_df[feature_cols + [target_col]].values


        X_train_seq, y_train_seq = create_sequences_for_lstm(train_df,
feature_cols, target_col=target_col, seq_len=seq_len)

        X_test_seq, y_test_seq = create_sequences_for_lstm(test_df,
feature_cols, target_col=target_col, seq_len=seq_len)


        # Rescale feature dimensions inside sequences using the previously fit
scaler

        n_features = X_train_seq.shape[2]


        def scale_seq(X_seq, scaler):

            flat = X_seq.reshape(-1, n_features)

            flat_scaled = scaler.transform(flat)

            return flat_scaled.reshape(X_seq.shape)


        X_train_seq_s = scale_seq(X_train_seq, scaler)

        X_test_seq_s = scale_seq(X_test_seq, scaler)


        # Build and train LSTM

        print("[INFO] Building LSTM model...")

        lstm = build_lstm_model((X_train_seq_s.shape[1],
X_train_seq_s.shape[2]))

        es = EarlyStopping(monitor='val_loss', patience=3,
restore_best_weights=True, verbose=1)
```

```
        history = lstm.fit(X_train_seq_s, y_train_seq,
validation_data=(X_test_seq_s, y_test_seq), epochs=10, batch_size=32,
callbacks=[es], verbose=1)


        y_pred_lstm = lstm.predict(X_test_seq_s).flatten()

        lstm_metrics = compute_metrics(y_test_seq, y_pred_lstm)

        print("[RESULT] LSTM metrics:", lstm_metrics)


        # Save LSTM

        lstm.save(os.path.join(output_dir, "lstm_energy_model"))

        print("[INFO] Saved LSTM model in", os.path.join(output_dir,
"lstm_energy_model"))


        # Plot LSTM results (compare only aligned lengths)

        plot_actual_vs_pred(y_test_seq, y_pred_lstm, "Actual vs Predicted
(LSTM)", out_path=os.path.join(output_dir, "lstm_actual_vs_pred.png"))

        plot_error_histogram(y_test_seq, y_pred_lstm,
out_path=os.path.join(output_dir, "lstm_error_hist.png"))

    else:

        if use_lstm and not TF_AVAILABLE:

            print("[WARN] TensorFlow not installed; skipping LSTM training.")


    # Optimization recommendations based on RF predictions (simple rules)

    recs = optimization_recommendation(y_pred_rf, threshold_factor=1.20)

    # Save sample recommendations CSV

    rec_df = pd.DataFrame(recs)

    rec_df.to_csv(os.path.join(output_dir, "optimization_recommendations.csv"),
index=False)

    print("[INFO] Saved optimization recommendations (sample) in",
os.path.join(output_dir, "optimization_recommendations.csv"))


    print("[INFO] Pipeline complete. Outputs saved in:", output_dir)

    return {

        'rf_metrics': rf_metrics,

        'lstm_metrics': lstm_metrics,

        'models': {'rf': os.path.join(output_dir, "rf_energy_model.joblib"),

                   'scaler': os.path.join(output_dir, "scaler.joblib"),
```

```
                    'lstm': os.path.join(output_dir, "lstm_energy_model") if
(use_lstm and TF_AVAILABLE) else None}
    }



# ----------------------------
# 11) Inference helper functions
# ----------------------------
def load_models(rf_path, scaler_path, lstm_path=None):
    rf = joblib.load(rf_path)
    scaler = joblib.load(scaler_path)
    lstm = None
    if lstm_path and TF_AVAILABLE and os.path.exists(lstm_path):
        lstm = tf.keras.models.load_model(lstm_path)
    return rf, scaler, lstm



def predict_single(rf_model, scaler, feature_vector):
    """
    feature_vector: 1D array of raw (unscaled) feature values matching training
order
    """
    fv = np.array(feature_vector).reshape(1, -1)
    fv_scaled = scaler.transform(fv)
    pred = rf_model.predict(fv_scaled)
    return float(pred[0])
```

```
# ----------------------------
# 12) CLI entry
# ----------------------------
def parse_args():
    parser = argparse.ArgumentParser(description="Run energy forecasting pipeline.")
    parser.add_argument("--csv", type=str, default="power_data.csv", help="Path to power CSV (optional).")
    parser.add_argument("--no-lstm", action="store_true", help="Skip LSTM even if TensorFlow present.")
    parser.add_argument("--out", type=str, default="outputs", help="Output directory.")
    return parser.parse_args()



if _name_ == "_main_":
    args = parse_args()
    run_pipeline(csv_path=args.csv, use_lstm=(not args.no_lstm), output_dir=args.out)
```